BIT 2118: Scientific Computing - Project Explanations

This document explains the mathematical principles and the real-world applications solved in Tasks 2 through 20 of the Scientific Computing project.

Task 2: NumPy Arrays and Numerical Computation

The Principle: Numerical computation relies on vectorization—applying mathematical operations to entire arrays of data at once rather than looping through individual elements. This takes advantage of low-level, highly optimized C code inside libraries like NumPy.

What is Being Solved: We processed 24 hours of greenhouse humidity data. Instead of checking each hour one by one, the code instantly calculates the daily mean, minimum, and maximum, and scales the data between 0 and 1 so it can be fed into an automated Machine Learning climate control system.

Task 3: Mathematical Functions and Vectorization

The Principle: Vectorization bypasses the overhead of standard Python loops. By executing operations on contiguous memory blocks, the CPU can use SIMD (Single Instruction, Multiple Data) to process data in parallel.

What is Being Solved: We simulated processing 1,000,000 e-commerce transactions to calculate taxes and final totals. The script demonstrates that using NumPy vectors is orders of magnitude faster (e.g., 45x speedup) than calculating the tax for each transaction sequentially in a loop.

Task 4: Floating-Point Arithmetic and Numerical Errors

The Principle: Computers represent real numbers in binary (base-2), which cannot perfectly represent fractions like 0.1, leading to tiny rounding errors. Furthermore, subtracting two nearly identical large numbers causes "catastrophic cancellation" (loss of significance).

What is Being Solved: We showed that summing 0.1 ten times does not exactly equal 1.0. More importantly, we solved a quadratic equation for a physics simulation using two different formulas. The standard formula failed due to precision loss, while our algebraically stabilized formula found the exact root, preventing the simulation from crashing.

Task 5: Matrix Algebra and Matrix Operations

The Principle: Matrices represent linear transformations in space. Multiplying a coordinate vector by a transformation matrix applies operations like scaling, rotation, or shearing. The determinant of this matrix reveals how the overall area or volume scales during the transformation.

What is Being Solved: We applied scale and shear matrices to the 2D coordinates of a bounding box. In computer vision, this technique is used to normalize the perspective of a detected object (like a skewed license plate) before passing it to a neural network for classification.

Task 6: Solution of Systems of Linear Equations

The Principle: Real-world constraints can often be modeled as systems of linear equations (Ax = b). Using matrix algebra, we can invert the coefficient matrix 'A' to find the exact combination of variables 'x' that satisfies all constraints 'b'.

What is Being Solved: We solved a factory production planning problem. Given a strict limitation on available labor hours and raw wood, the linear solver calculated the exact number of chairs and tables the factory must produce to completely maximize resource usage with zero waste.

Task 7: Gaussian Elimination and LU Decomposition

The Principle: Solving Ax = b repeatedly for different 'b' vectors is computationally expensive. LU Decomposition breaks the matrix 'A' into Lower (L) and Upper (U) triangular matrices once. We can then solve for any 'b' using simple forward and backward substitution.

What is Being Solved: We modeled a bridge's structural nodes (the stiffness matrix K). Since the bridge itself doesn't change, we factored K once. We then instantly calculated how much the bridge bends (displacement) when different vehicles (load vectors) drive across it.

Task 8: Iterative Methods for Linear Systems

The Principle: For massive, sparse matrices (mostly zeros), direct solving (like LU decomposition) takes too much memory. Iterative methods like Jacobi and Gauss-Seidel start with a guess and repeatedly refine it until the error drops below a specific tolerance.

What is Being Solved: We modeled the steady-state diffusion of chemical concentrations in a pipe connecting two reservoirs. The script iteratively refined the concentration at intermediate points until it reached equilibrium, showing that the Gauss-Seidel method converges to the answer faster than the Jacobi method.

Task 9: Eigenvalues and Eigenvectors

The Principle: For a given matrix, eigenvectors are vectors that only stretch (scale) when the matrix is applied to them, without changing direction. The eigenvalue is the amount they stretch. In dynamic systems, the dominant eigenvalue dictates long-term growth.

What is Being Solved: We used a Leslie Matrix to model the ecology of an animal population. By finding the matrix's eigenvalues, we determined the long-term growth rate of the species (shrinking or growing), and the eigenvector revealed the stable percentage of juveniles versus adults in the environment.

Task 10: Root Finding: Bisection and Fixed-Point Methods

The Principle: Root-finding algorithms find where a function equals zero (f(x) = 0). The Bisection method traps the root in a shrinking interval. The Fixed-Point method rearranges the equation to x = g(x) and loops x = g(x) until it stabilizes.

What is Being Solved: We analyzed a manufacturing business to find the exact break-even point where revenue perfectly offsets nonlinear costs (Profit = 0). The code identified the required production volume using both methods and plotted the intersection graph.

Task 11: Root Finding: Newton-Raphson and Secant Methods

The Principle: These are fast, derivative-based root finders. Newton-Raphson uses the exact mathematical slope (derivative) to rapidly slide down the curve to the root. The Secant method approximates this slope using two recent points when the exact derivative is too hard to calculate.

What is Being Solved: We calculated the volume of Carbon Dioxide gas using the non-linear Van der Waals equation of state. Unlike the ideal gas law, this incorporates complex intermolecular forces, requiring advanced root-finding to discover the true physical volume.

Task 12: Polynomial and Nonlinear Equation Solving

The Principle: Real systems often involve multiple variables interacting non-linearly (e.g., trigonometric functions or squares). Numerical solvers use multi-dimensional algorithms (like the Newton-Krylov method) to find the set of variables that make all equations true simultaneously.

What is Being Solved: We performed Inverse Kinematics for a robotic arm. Given a desired (x, y) target coordinate for the robot's hand, the solver calculated the exact angles the two joint motors must rotate to in order to reach that specific coordinate.

Task 13: Interpolation

The Principle: Interpolation estimates unknown data points that fall between known data points. Linear interpolation draws straight lines between points. Spline interpolation fits piecewise polynomial curves (like cubics) to ensure the transitions are perfectly smooth and continuous.

What is Being Solved: We tackled a computer animation problem. Given only a few sparse "keyframes" of an object's trajectory, the cubic spline interpolation smoothly filled in the missing frames, ensuring fluid, natural motion at high frame rates.

Task 14: Curve Fitting and Approximation

The Principle: Curve fitting (Regression) finds the optimal mathematical parameters that make a theoretical curve best match messy, real-world data. It minimizes the squared differences (residuals) between the curve and the data points.

What is Being Solved: We modeled biological bacteria growth. The algorithm analyzed lab samples over time and successfully reverse-engineered the underlying exponential growth rate parameter, allowing us to predict future population spikes.

Task 15: Numerical Differentiation

The Principle: Differentiation measures the instantaneous rate of change (the slope). In discrete numerical data, we approximate this using finite differences—comparing a data point to its immediate neighbors divided by the time gap.

What is Being Solved: We built a financial momentum indicator. By applying numerical differentiation to daily stock prices, the script calculated the speed of price changes (momentum), a metric used by algorithmic trading systems to trigger buy or sell signals.

Task 16: Numerical Integration

The Principle: Integration calculates the total accumulated quantity (the area under a curve). The Trapezoidal rule calculates this area using linear segments, while Simpson's rule uses parabolic arcs, which is vastly more accurate for smooth, continuous data.

What is Being Solved: We analyzed hydrological data at a dam. By integrating the highly variable water flow rates (cubic meters per second) over a 12-hour period, we precisely calculated the total massive volume of water discharged.

Task 17: Ordinary Differential Equations – Euler Method

The Principle: An ODE describes a system's rate of change over time. The Euler method is a step-by-step numerical approach: it looks at the current state, calculates the current rate of change, and takes a small step forward in time to predict the next state.

What is Being Solved: We modeled the ecological logistic growth of a species. Unlike unchecked exponential growth, this ODE factors in environmental limits (carrying capacity). The simulation mapped out the population's classic 'S-curve' as it grew and eventually stabilized.

Task 18: Numerical Solution of ODEs Using SciPy

The Principle: Basic methods like Euler accumulate error rapidly. Professional solvers (like those in SciPy) use advanced algorithms (e.g., Runge-Kutta) that dynamically adjust their time-steps to maintain high accuracy without sacrificing speed.

What is Being Solved: We modeled a viral disease outbreak using the epidemiological SIR (Susceptible, Infected, Recovered) model. SciPy perfectly integrated the three interacting equations, predicting exactly when the infection wave would peak and how many people would be affected.

Task 19: Numerical Optimization

The Principle: Optimization seeks to find the absolute minimum or maximum of an objective function. Constrained optimization (like the SLSQP algorithm) does this while strictly adhering to boundaries and rules (inequalities) that the variables must not violate.

What is Being Solved: We solved a Supply Chain Logistics problem. The algorithm determined exactly how many production hours to allocate to two different factories to completely minimize manufacturing costs, all while mathematically guaranteeing a minimum market demand was met.

Task 20: Random Numbers and Monte Carlo Methods

The Principle: Monte Carlo methods solve deterministic problems using randomness. By running tens of thousands of simulated scenarios with varying inputs drawn from statistical distributions, we can empirically estimate the probability of complex outcomes.

What is Being Solved: We performed a PERT (Project Evaluation and Review Technique) analysis. By simulating the durations of three sequential project tasks 100,000 times, we calculated the exact percentage chance that the highly unpredictable project would meet its strict 50-day deadline.